

Fun with Robots

Projekt-Einzelbericht

Autor: Tilo Brockmann

Matrikelnummer: 3697647

Studiengang: Media Engineering

Semester: 6

Technische Hochschule Nürnberg Georg Simon Ohm
Fakultät Elektrotechnik Feinwerktechnik Informationstechnik

15. August 2025

Selbstständigkeitserklärung

Hiermit erkläre ich, Tilo Brockmann, dass die vorliegende Arbeit selbstständig und ohne fremde Hilfe (Dritter) verfasst und keine anderen Hilfsmittel als die im Literaturverzeichnis angegebenen Quellen verwendet wurden. Insbesondere versichern wir, dass wir alle wörtlichen und sinngemäßen Übernahmen aus anderen Werken als solche kenntlich gemacht haben.

Ort, Datum

Unterschrift

Inhaltsverzeichnis

1	Einleitung	2
2	Recherche	3
2.1	Werkstoffe und Alternativen zum 3D-Druck	3
2.2	MQTT	3
3	Robotersteuerung	5
3.1	Anforderungen	5
3.2	EasyMQTT	6
3.2.1	Planung	6
3.2.2	Implementierung	7
	ID Generierung:	7
	Konstruktor und grundlegende Nutzung:	7
	Registrierung:	9
	Claim- und Locking-Mechanismus (Handshake):	10
	Quality-of-Life-Funktionen (Wrapper und Hilfsmethoden)	11
	Implementierung in der Robotersteuerung	12
3.3	Webinterface	14
3.3.1	Planung	14
3.3.2	Implementierung	15
	Aufruf der Seite:	15
	Kommunikation mit dem Broker:	16
	Dynamische Anpassung der Seite:	17
3.4	Netzwerk und Hardware	19
4	Fazit	20
	Literaturverzeichnis	21

1 Einleitung

Während im vorangegangenen Semester der Schwerpunkt überwiegend auf Einzelarbeit lag, stand das aktuelle Semester stärker im Zeichen von Arbeitsteilung und teamorientierter Zusammenarbeit. Die Recherchearbeiten wurden zwischen den Teammitgliedern aufgeteilt, wie im Gesamtbericht detailliert dargestellt. Im praktischen Teil entstanden gemeinsam sowohl unterschiedliche Robotermodelle mit spezifischen Funktionen als auch mehrere baugleiche Exemplare, um unter anderem die hierfür benötigte Zeit zu ermitteln und den Ablauf zu optimieren.

Der persönliche Schwerpunkt lag auf der Entwicklung der Systemarchitektur im Hintergrund, insbesondere auf der Implementierung der Steuerungs- und Kommunikationslogik. Diese umfasste die Ausarbeitung einer skalierbaren Backend-Struktur, die standardisierte Anbindung der Roboter an das Netzwerk sowie die Umsetzung einer zuverlässigen, plattformunabhängigen Steuerungsmöglichkeit.

2 Recherche

2.1 Werkstoffe und Alternativen zum 3D-Druck

Der 3D-Druck hat sich als flexibles Fertigungsverfahren etabliert, ist jedoch insbesondere für die Herstellung mehrerer identischer Roboterkomponenten nicht immer sinnvoll einsetzbar. Die begrenzte Fertigungsgeschwindigkeit pro Bauteil verzögert die Produktion größerer Stückzahlen, und kostspielige 3D-Druckanlagen sind für Privatpersonen und Bildungseinrichtungen oft nicht zugänglich – beides widerspricht dem Ziel eines niederschweligen „Fun with Robots“-Ansatzes.

Untersuchte Werkstoffe im Überblick:

- **Holzwerkstoffe (z. B. Sperrholz)** – Stabil, kostengünstig, gut laserschneidbar.
- **Karton / Pappe** – Leicht, günstig, präzise per Lasercutter bearbeitbar, umweltfreundlich.
- **Schichtaufbau (Holz + Karton)** – Kombination für erhöhte Stabilität.
- **Technische Kunststoffe (z. B. Polypropylen, Acryl, POM)** – Hohe Festigkeit, jedoch teurer.

Für viele Anwendungsfälle erwies sich Karton aufgrund seiner Kosteneffizienz, Verfügbarkeit und einfachen Handhabung als besonders geeignet. Durch Verwendung eines Lasercutters konnte Karton präzise zugeschnitten und mit Perforierungen versehen werden, um passgenaue und reproduzierbare Bauteile herzustellen – ohne aufwendige Werkzeuge oder Formen.

2.2 MQTT

Im Rahmen unserer Ziele für die zweiten Projektphase haben wir uns mit verschiedenen Möglichkeiten der Kommunikation zwischen unseren Robotern beschäftigt. Dabei sind wir auf das MQTT-Protokoll[1] gestoßen – eine weit verbreitete Lösung für die Machine-to-Machine-Kommunikation.

MQTT steht für *Message Queuing Telemetry Transport* und ist ein Protokoll, das ursprünglich für Geräte mit geringer Bandbreite oder instabiler Netzwerkverbindung entwickelt wurde. Es eignet sich besonders gut für Anwendungen, bei denen viele Geräte Daten miteinander austauschen sollen, ohne dabei große Datenmengen zu übertragen oder komplexe Kommunikationsstrukturen aufzubauen.

Das Grundprinzip von MQTT basiert auf dem sogenannten Publish-Subscribe-Modell. Dabei gibt es einen zentralen Vermittler, den sogenannten „Broker“. Die Roboter können entweder Nachrichten an bestimmte Themen („Topics“) senden (*Publish*) oder sich für diese Themen anmelden (*Subscribe*). Ein Roboter könnte z. B. eine Nachricht mit dem Inhalt „Fahrt starten“ an das Topic `roboter/gruppe1/start` senden, während alle anderen Roboter, die dieses Topic abonniert haben, sofort reagieren können. Der Vorteil dieses Systems liegt in seiner Einfachheit und Effizienz: Geräte müssen sich nicht direkt kennen

oder miteinander verbunden sein. Alles läuft über den Broker, der die Nachrichten verwaltet und verteilt. Besonders sinnvoll ist MQTT dann, wenn mehrere Geräte gleichzeitig gesteuert, koordiniert oder mit Informationen versorgt werden sollen – so wie es in unserer Projektidee mit einer vernetzten Robotergruppe der Fall ist.

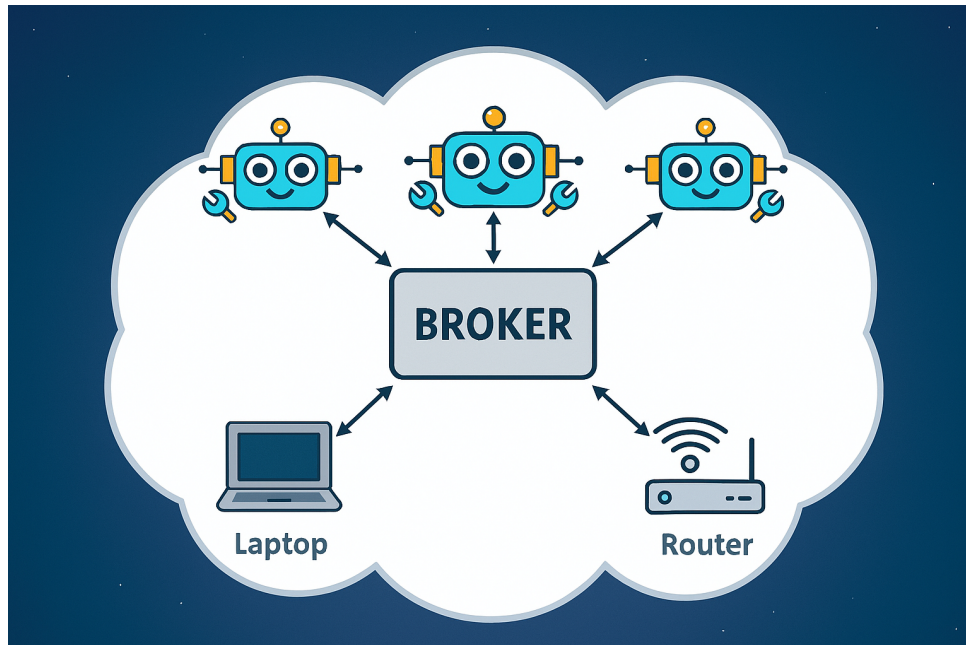
Ein weiterer Pluspunkt ist die Plattformunabhängigkeit. MQTT funktioniert mit nahezu allen Programmiersprachen und Geräten. Für Mikrocontroller wie den ESP32 oder Arduino lassen sich passende Bibliotheken verwenden, um mit wenigen Zeilen Code eine Verbindung zum Broker herzustellen.

Auch bei der Wahl des Brokers gibt es viele Möglichkeiten – von cloudbasierten Anbietern bis hin zu lokalen Lösungen, wie z. B. dem Open-Source-Broker „Mosquitto“[2], den man auf einem Raspberry Pi oder einem herkömmlichen PC oder Mac betreiben kann. Für unser Projekt könnte ein lokaler Broker die beste Lösung sein, da er unabhängig vom Internet funktioniert und keine zusätzlichen Online-Dienste benötigt.

Insgesamt lässt sich sagen, dass MQTT eine einfache, zuverlässige und gut dokumentierte Lösung für die Kommunikation zwischen Robotern darstellt – besonders dann, wenn mehrere Geräte gleichzeitig miteinander interagieren sollen. Die nächsten Schritte bestanden nun darin, erste Testverbindungen zwischen zwei Robotern über MQTT herzustellen und auszuloten, wie sich die Koordination mehrerer Roboter praktisch umsetzen lässt

3 Robotersteuerung

3.1 Anforderungen



Der Nachbau und die Bedienung der entwickelten Roboter sollen möglichst barrierearm gestaltet werden. Zwar ermöglicht das MQTT-Protokoll eine flexible und plattformunabhängige Kommunikation zwischen Steuerungseinheiten und Robotern, dennoch ergeben sich zwei zentrale Herausforderungen.

Zum einen erfordert die zuverlässige Steuerung über MQTT auch ohne den Einsatz eines eigenen TCP-Protokolls eine beträchtliche Menge nicht-trivialen Codes pro Roboter. Mit steigender Anzahl an Robotern erhöht sich die Komplexität weiter, was dem angestrebten „Fun with Robots“-Ansatz entgegenwirkt. Zur Reduzierung dieses Aufwands wurde die Arduino-Bibliothek *EasyMQTT* entwickelt. Sie standardisiert die Kommunikation mit dem Broker, stellt automatisch eine Verbindung zu WLAN und Broker her und integriert mehrere Komfortfunktionen. Dadurch entfällt für Anwenderinnen und Anwender weitgehend die Auseinandersetzung mit den technischen Details von MQTT. Das zugrunde liegende Prinzip lautet: *Konfiguration statt Programmierung*.

Zum anderen stellt die direkte Steuerung eine weitere Hürde dar. Werkzeuge wie der MQTT-Explorer sind zwar geeignet, um den Nachrichtenverkehr zu analysieren, erlauben jedoch lediglich das manuelle Senden einzelner Nachrichten an spezifische Topics. Ein kontinuierlicher Datenstrom an Steuereingaben ist auf diesem Weg nicht realisierbar. Für eine flexible und plattformunabhängige Steuerung wurde daher eine webbasierte Benutzeroberfläche entwickelt, die sich dynamisch an die jeweiligen Funktionen eines Roboters anpasst und sowohl auf Smartphones als auch auf Desktop-Computern genutzt werden kann.

3.2 EasyMQTT

3.2.1 Planung

Vor dem Beginn der Entwicklung, erfolgte eine detaillierte Planungsphase. Wie schon erwähnt hat jeder Roboter individuelle Funktionen. Daraus folgt, dass bei der Entwicklung der Bibliothek ein modularer Ansatz verfolgt werden musste. Ziel war es, eine einheitliche Kommunikationsstruktur zwischen mehreren Robotern verschiedener Typen und den zugehörigen Steuerclients zu schaffen.

Der Planungsprozess umfasste die Definition einer klaren MQTT-Topic-Struktur, die die Registrierung, Statusübermittlung und Steuerung der Roboter eindeutig voneinander trennt. Für die Identifikation wurde entschieden, einen festen Bestandteil der MAC-Adresse als eindeutige Kennung zu verwenden. Dies gewährleistet, dass jeder Roboter auch nach einem Neustart oder einer Neuverbindung im System unverwechselbar bleibt. Zudem ist es so möglich baugleiche Roboter zu differenzieren. Damit ergibt sich für jeden Roboter ein Wurzelverzeichnis nach dem Schema:

```
<Roboter-Typ>/<Roboter-ID>/#
```

Darüber hinaus wurde ein Registrierungs- und Statusprotokoll spezifiziert, bei dem die Roboter beim Start automatisch eine retained Nachricht mit ihren Identifikations- und Typinformationen an den Broker senden. Zusätzlich werden in regelmäßigen Abständen Heartbeat-Nachrichten übermittelt, um den Betriebszustand aktuell zu halten.

```
<Roboter-Typ>/<Roboter-ID>/register  
<Roboter-Typ>/<Roboter-ID>/status
```

Für die Steuerung wurde ein Claim- und Locking-Mechanismus vorgesehen, um sicherzustellen, dass jeweils nur ein Client gleichzeitig Befehle an einen bestimmten Roboter senden kann. Dieser Mechanismus beinhaltet einen Handshake-Prozess zwischen Client und Roboter sowie ein automatisches Freigeben des Zugriffs nach einer definierten Zeitspanne ohne Eingaben.

```
<Roboter-Typ>/<Roboter-ID>/claim  
<Roboter-Typ>/<Roboter-ID>/acknowledge
```

Die tatsächliche Steuerung erfolgt über individuelle Topics für jede mögliche Aktion:

```
<Roboter-Typ>/<Roboter-ID>/action/<Aktionsname>
```

Die Bibliotheksarchitektur wurde so konzipiert, dass die Roboterseite auf WiFi-fähigen Mikrokontrollern mit der Arduino-IDE und PubSubClient[3] arbeitet.

3.2.2 Implementierung

ID Generierung:

In der praktischen Umsetzung erwies sich die Nutzung der MAC-Adresse als unzuverlässig, da in vielen Fällen ein Dummy-Wert zurückgegeben wurde, was zu ID-Duplikaten führen konnte. Um eine robuste, konsistente und dennoch einfache Möglichkeit zur eindeutigen Identifikation zu schaffen, wurde auf die interne Seriennummer des Mikrocontrollers zurückgegriffen. Dabei werden gezielt drei Bytes der Seriennummer extrahiert und zu einer eindeutigen Kennung zusammengesetzt:

```
const char* _getUniqueId() {
    static char out[16];

    // Retrieve Serial Number
    uint32_t w0 = *(uint32_t*)0x0080A00C;
    uint32_t w1 = *(uint32_t*)0x0080A040;
    uint32_t w2 = *(uint32_t*)0x0080A044;
    uint32_t w3 = *(uint32_t*)0x0080A048;

    // Use last three bytes as probably unique identifier
    uint8_t b0 = (w0 >> 16) & 0xFF;
    uint8_t b1 = (w1 >> 8) & 0xFF;
    uint8_t b2 = w2 & 0xFF;

    sprintf(out, "Robot-%02X%02X%02X", b0, b1, b2);
    return out; // z.B. "Robot-5F0F8A"
}
```

Funktionsweise:

- Die Funktion liest an festen Speicheradressen die im Mikrocontroller hinterlegte Seriennummer aus.
- Aus den vier gelesenen 32-Bit-Werten werden drei Bytes selektiert, die mit an Sicherheit grenzender Wahrscheinlichkeit pro Gerät einzigartig sind.
- Diese Bytes werden als Hex-Werte in einen String der Form Robot-XXXXXX formatiert.
- Die statische Variable out speichert das Ergebnis dauerhaft, sodass der Rückgabewert auch nach Verlassen der Funktion gültig bleibt.

Konstruktor und grundlegende Nutzung:

Die Bibliothek erfordert bereits bei der Instanziierung alle relevanten Parameter, um Konfigurationsaufwand im späteren Code zu minimieren. Dazu zählen WLAN-Zugangsdaten, Broker-IP und -Port, der Gerätetyp sowie eine Liste unterstützter Aktionen, die intern auf MAX_ACTIONS begrenzt werden. Der Konstruktor speichert diese Werte und initialisiert interne Zustandsvariablen:

```

EasyMQTT(  const char*  ssid,
           const char*  pass,
           IPAddress     brokerIP,
           uint16_t      brokerPort,
           const char*   deviceType,
           const char*   actions[],
           const size_t  actionCount
) : _ssid(ssid), _pass(pass),
    _brokerIP(brokerIP), _brokerPort(brokerPort),
    _deviceType(clientType), _mqtt(_wifiClient),
    _isInUse(false), _last(0), _lastPingFromClient(0)
{
    _actionCount = min(actionCount, MAX_ACTIONS);
    for (size_t i = 0; i < _actionCount; ++i) {
        _actions[i] = actions[i];
    }
}

```

Das Objekt wird außerhalb von `setup()` global erstellt. In `setup()` werden die Verbindung und Registrierung mit `begin()` gestartet sowie der Callback für eingehende Nachrichten gesetzt:

```

void begin() {
    if (!Serial.available()) {
        delay(1000);
    }

    _connectWiFi();
    _deviceId = _getUniqueId();
    _topicPrefix = String(_deviceType) + "/" + String(_deviceId) + "/";
    _mqtt.setServer(_brokerIP, _brokerPort);
    _connectMQTT();
    _registerDevice();
}

```

Für die Verbindung mit dem Netzwerk wurde die `WiFi101`[4] Bibliothek verwendet. In `loop()` wird `mqtt_loop()` zyklisch aufgerufen, um Verbindung und Status aktuell zu halten:

```

void mqtt_loop() {
    if (!_mqtt.connected()) {
        Serial.print("connection to broker lost, ");
        _connectMQTT();
    }
    if (_elapsed(5000)) {
        _sendHeartBeat();
    }
    if (_hasTimeoutOccoured()) {
        resetControllerId();
    }
    _mqtt.loop();
}

```

Zusätzlich sendet die Bibliothek in regelmäßigen Abständen einen Heartbeat auf dem Statustopic <Gerätetyp>/<Geräte-ID>/status. Die Nachricht wird als *retained* veröffentlicht, sodass neu verbundene Clients den aktuellen Zustand sofort erhalten. Der Heartbeat ist als JSON[5] strukturiert, z. B.:

```

{
  "online": true,
  "locked": false
}

```

Dabei kennzeichnet "online" die grundsätzliche Erreichbarkeit des Roboters über den Broker, während "locked" anzeigt, ob eine exklusive Steuerverbindung (Claim) aktiv ist (**true**) oder der Roboter frei ist (**false**). Geht der Roboter unerwartet offline, setzt der beim Verbindungsaufbau konfigurierte MQTT *Last Will and Testament* auf demselben Topic automatisch "online": **false**, sodass der Ausfall ohne zusätzliche Logik bei allen Clients sichtbar wird.

Registrierung:

Die Registrierung eines Geräts beim MQTT-Broker erfolgt in der privaten Methode `_registerDevice()`. Sie wird im Rahmen der Initialisierung von `begin()` aufgerufen und dient dazu, dem Broker strukturierte Informationen über den Roboter zur Verfügung zu stellen. Zunächst wird überprüft, ob eine aktive MQTT-Verbindung besteht. Anschließend wird mit der ArduinoJson[6]-Bibliothek ein JSON-Dokument erstellt, das die eindeutige Geräte-ID, den Gerätetyp sowie eine Liste aller unterstützten Aktionen enthält:

```

bool _registerDevice() {
    if (!_mqtt.connected()) {
        Serial.println("Device registration failed: Client not connected
            !");
    }
    StaticJsonDocument<256> doc;

    doc["id"] = _deviceID;
    doc["type"] = _deviceType;

    JsonArray actionsArray = doc.createNestedArray("actions");
    for (size_t i = 0; i < _actionCount; i++) {
        actionsArray.add(_actions[i]);
    }

    char payload[512];

    serializeJson(doc, payload);

    // Register Device
    Serial.println("Device registration: " + String(payload));
    return publish("register", payload, true);
}

```

Die erstellte JSON-Struktur wird anschließend serialisiert und als **retained** Nachricht auf dem Topic `<Typ>/<ID>/register` veröffentlicht. Dadurch ist die Information für neu verbundene Clients sofort verfügbar, ohne dass der Roboter erneut senden muss. Ein typisches Registrierungsobjekt kann beispielsweise wie folgt aussehen:

```

{
    id: "Robot-1f21",
    type: "Simplebot",
    actions: ["dir-forward", "dir-left", "dir-right"]
}

```

Claim- und Locking-Mechanismus (Handshake):

Damit jeweils nur ein Client exklusiv Befehle an einen Roboter senden kann, implementiert EasyMQTT ein leichtgewichtiges Claim-/Lock-Protokoll. Ein Client fordert die Kontrolle an, indem er seine eindeutige Kennung (Controller-ID) auf `<Typ>/<ID>/control/claim` publiziert. Ist der Roboter frei (`_isInUse = false`), wird die Verbindung angenommen, die ID zwischengespeichert und ein `"true"` als Bestätigung auf `<Typ>/<ID>/control/acknowledge` gesendet. Anschließend gilt der Roboter als *locked*.

```

bool manageConnection(String topic, String payload) {
    // Handle Connection
    if (!_isInUse) {
        // Await connection
        if (topic == _topicPrefix + "control/claim" && payload != "") {
            Serial.println("Client " + payload + " claims the connection");
            _acknowledgeConnection(payload);
            return true;
        }
        // No handshake in progress
        return false;
    }

    // Measure time since last ping
    if (topic == _topicPrefix + "ping" && payload == _controllerID) {
        _lastPingFromClient = millis();
    }
    return true;
}

```

Die eigentliche Bestätigung (Acknowledge) setzt den internen Zustand auf „in Benutzung“ und startet den Timeout-Timer. Die Rückmeldung erfolgt auf dem Acknowledge-Topic:

```

void _acknowledgeConnection(String controllerID) {
    _controllerID = controllerID;
    _isInUse = true;
    _lastPingFromClient = millis();

    publish("control/acknowledge", "true");
}

```

Lebenszeichen (Ping) und Timeout: Solange ein Client exklusiv verbunden ist, muss er regelmäßig auf `<Typ>/<ID>/ping` seine Controller-ID senden. Jeder gültige Ping aktualisiert den Zeitstempel `_lastPingFromClient`. Bleibt ein Ping oder eine Aktion länger als fünf Sekunden aus, wird die Verbindung automatisch freigegeben; der Roboter ist wieder anforderbar (*unlock*). Der aktuelle Sperrstatus spiegelt sich zudem im Heartbeat wider (vgl. Status-Topic `.../status` mit `"locked": true/false`).

Quality-of-Life-Funktionen (Wrapper und Hilfsmethoden)

Neben der Kernlogik stellt EasyMQTT eine Reihe von Wrappern und Hilfsmethoden bereit, die typische MQTT-Aufgaben vereinfachen und Boilerplate reduzieren. Zentrale Entwurfsentscheidung ist die konsequente Verwendung von `String` (Arduino) statt `char*` in der öffentlichen API sowie das automatische Voranstellen eines gerätespezifischen Topic-Präfixes. Dadurch lassen sich Fehlerquellen reduzieren und der Code bleibt lesbar.

- **subscribe(String topicSuffix):** Abonniert Topics relativ zum Gerätepräfix, z. B. `subscribe("action/forward")` statt des vollständigen Pfads `<Typ>/<ID>/action/forward`.

Vorteil: Weniger Tippfehler, leichteres Refactoring bei Typ-/ID-Änderungen.

- **publish(String topicSuffix, String payload, bool retained=false)**: Veröffentlicht Nachrichten auf einem Suffix-Topic (inkl. optional *retained*) und loggt den Versand. Vorteil: Kein manuelles `c_str()`, konsistente Retain-Nutzung, saubere Trennung von Daten und Transport.
- **getDeviceID()** und **getTopicPrefix()**: Komfort für Logging/Debugging; vermeidet Redundanz beim manuellen Aufbau von Topic-Strings.
- **isInUse()**: Fragt den Lock-Status ab.
- **manageConnection(topic, payload)**: Kapselt den Claim-Handshake und Ping-Erhalt (siehe eigener Abschnitt). Vorteil: Minimale Zustandslogik im Anwendercode.
- **clientMatches(actionPayload)**: Prüft, ob eine eingehende Aktion zur aktuell verbundenen Controller-ID gehört. Erwartetes Format: `<id>:<msg>`. Vorteil: Kein manuelles Parsen im Callback.
- **payloadContentMatches(actionPayload, expected)**: Extrahiert den optionalen Nachrichtenteil (`<msg>`) und vergleicht ihn mit `expected`. Vorteil: Schnelle Intent-Prüfung ohne JSON.
- **convertPayloadToString(uint8_t*, len)**: Konvertiert rohe MQTT-Payloads aus dem Callback in einen `String`. Vorteil: Vermeidet wiederkehrende Schleifen/Fehler bei der Umwandlung.

Implementierung in der Robotersteuerung

In der praktischen Umsetzung wird die Bibliothek direkt in den Hauptablauf (`loop()`) und in den MQTT-Callback integriert. Die Steuerlogik folgt einem einfachen Zustandsprinzip: Die Variable `currentAction` speichert den aktuell auszuführenden Fahrbefehl, der im `switch`-Block in konkrete Bewegungsfunktionen übersetzt wird. Jede erkannte Eingabe (z. B. „Taste gedrückt“) setzt diesen Zustand, und ein „Button-up“-Signal setzt ihn wieder auf Null, wodurch der Roboter stoppt.

```
void loop() {
    bot.mqtt_loop();

    switch (currentAction) {
        case 1: driveForward(); break;
        case 2: turnLeft();      break;
        case 3: turnRight();     break;
        default: stopDrive();    break;
    }
    delay(60); // simple rate-limit
}
```

Im MQTT-Callback übernimmt `manageConnection()` den Claim-Handshake und prüft gemeinsam mit `clientMatches()`, ob die Nachricht vom aktuell verbundenen Client stammt. Anschließend werden Topics und Payloads mit den QoL-Methoden der Bibliothek (`getTopicPrefix()`, `payloadContentMatches()`) ausgewertet, um `currentAction` entsprechend zu setzen.

```

void mqttCallback(char* topic, uint8_t* payload, unsigned int length) {
    String topicStr    = String(topic);
    String prefix      = bot.getTopicPrefix();
    String payloadStr  = bot.convertPayloadToString(payload, length);

    if (!bot.manageConnection(topicStr, payloadStr) || !bot.clientMatches(
        payloadStr)) {
        return;
    }

    if (topicStr == prefix + "action/dir-forward" && bot.
        payloadContentMatches(payloadStr, "down")) currentAction = 1;
    else if (topicStr == prefix + "action/dir-left" && bot.
        payloadContentMatches(payloadStr, "down")) currentAction = 2;
    else if (topicStr == prefix + "action/dir-right" && bot.
        payloadContentMatches(payloadStr, "down")) currentAction = 3;
    else if (bot.payloadContentMatches(payloadStr, "up")) currentAction =
        0;
}

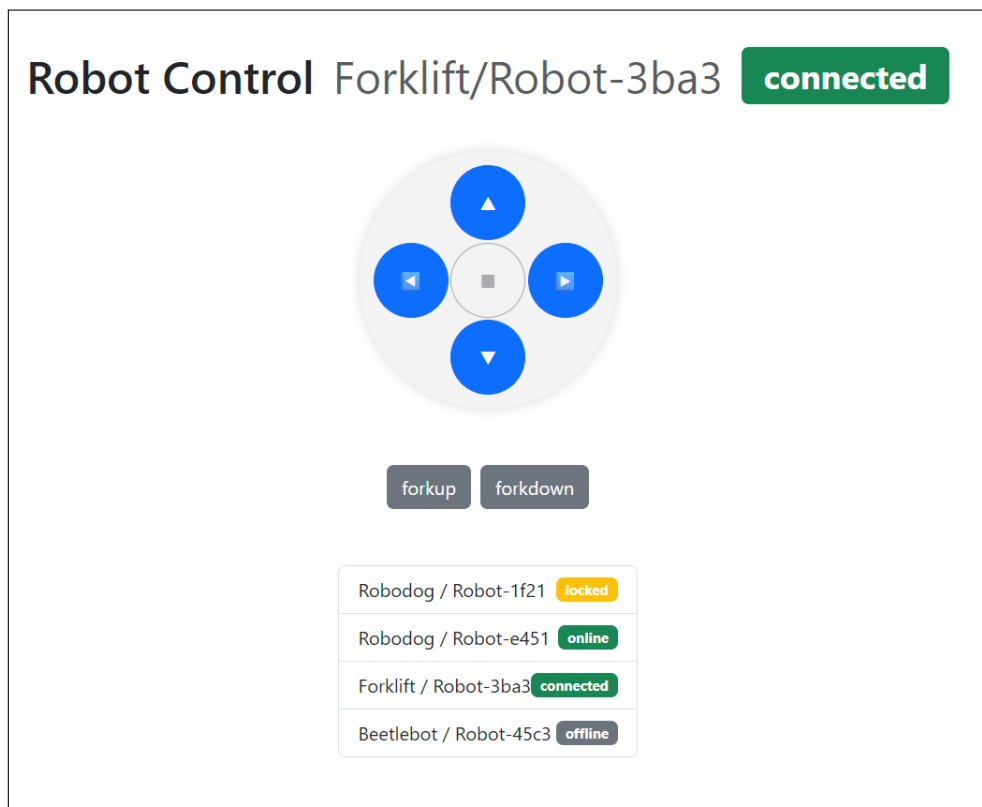
```

Durch die Nutzung von EasyMQTT bleibt die Implementierung auf wenige, klar lesbare Codezeilen beschränkt:

- Keine manuelle Verwaltung von `char*`-Zeigern oder komplexen Topic-Strings – stattdessen einfache `String`-Methoden mit automatischem Präfix.
- Integrierte Verbindungspflege, Claim- und Timeout-Logik, sodass im Sketch keine Low-Level-MQTT-Befehle notwendig sind.
- Klar strukturierte Trennung zwischen Kommunikationslogik (Bibliothek) und Bewegungslogik (Sketch).

Ohne die Bibliothek wäre eine vergleichbare Implementierung deutlich umfangreicher und fehleranfälliger, da Topic-Management, Verbindungsstatus, Heartbeat und Parsing manuell programmiert werden müssten. In dießer Form reichte häufig ein einziger Prompt in einem Large Language Model, um bestehenden auf Serieller Steuerung angewiesenen Robotercode auf die EasyMQTT Architektur umzustellen.

3.3 Webinterface



3.3.1 Planung

Für die Umsetzung einer plattformunabhängigen Steuerlösung fiel die Wahl auf eine Kombination aus HTML, JavaScript und dem CSS-Framework Bootstrap[7]. Für die Implementierung von MQTT wurde die Bibliothek MQTT.js[8] gewählt. Diese Technologien gewährleisten eine breite Kompatibilität, da sie in jedem modernen Browser – unabhängig von Betriebssystem oder Endgerät – nutzbar sind und gleichzeitig eine schnelle Entwicklung einer funktionsfähigen, responsiven Benutzeroberfläche ermöglichen. Bootstrap sorgt hierbei für ein automatisch an verschiedene Bildschirmgrößen angepasstes Layout und reduziert den manuellen Styling-Aufwand.

Analog zur Entwicklung der Bibliothek **EasyMQTT** musste auch beim Webinterface die große Diversität der Roboter berücksichtigt werden. Während die meisten Modelle über grundlegende Fahrfunktionen verfügen, existieren Unterschiede bei den verfügbaren Richtungen und Zusatzfunktionen. Das Interface wurde daher von Beginn an so geplant, dass es sich dynamisch an die registrierten Fähigkeiten eines Roboters anpasst. Die Steuerung sollte sowohl auf Touch-Geräten (Smartphones, Tablets) als auch auf Desktop-Systemen komfortabel bedienbar sein. Dazu wurden ein virtuelles, touch-fähiges D-Pad für mobile Geräte sowie die Unterstützung von Tastatureingaben (Pfeiltasten und WASD) für PCs vorgesehen.

Das D-Pad wird dynamisch konfiguriert: Nicht unterstützte Richtungen werden automatisch ausgegraut und deaktiviert. Zusatzfunktionen erscheinen als separate Schaltflä-

chen, die zentriert unter dem D-Pad angeordnet werden. Um diesen dynamischen Aufbau der GUI zu ermöglichen, wird auf ein einheitliches Namensschema für Aktionstopics gesetzt. Alle Steuerbefehle liegen unter dem Pfad `<Typ>/<ID>/action/#`, wobei sich die Art der Aktion aus der Benennung ergibt:

- Richtungsbefehle folgen dem Schema `dir-<Richtung>` (z.,B. `dir-forward`, `dir-left`).
- Zusatzfunktionen folgen dem Schema `extra-<Taste>-<Funktionsname>` (z.,B. `extra-e-shoot`).

Neben der Eingabeplanung war auch die Anzeige der im Netzwerk verfügbaren Roboter ein zentraler Punkt. Das Interface sollte alle aktiven Geräte mitsamt ihrem Online-Status und Sperrzustand (*online/locked/offline*) darstellen. Durch einfaches Antippen oder Anklicken sollte ein Claim-Prozess gestartet werden, um die exklusive Steuerung eines Roboters zu übernehmen. Die hierfür notwendige Claim-/Locking-Logik sowie der gesamte Handshake-Ablauf der EasyMQTT-Architektur mussten daher vollständig in die Webanwendung integriert werden.

Zur Kommunikation mit dem Broker wurde die JavaScript-Bibliothek `mqtt.min.js` eingebunden, während `bootstrap.min.css` für das Layout verantwortlich ist. Die gesamte Website wird lokal im Netzwerk gehostet, um geringe Latenzzeiten zu gewährleisten und keine Internetverbindung vorauszusetzen.

3.3.2 Implementierung

Aufruf der Seite:

Beim Laden der Webanwendung wird zunächst die MQTT-Clientinstanz erzeugt und eine Verbindung zum im Quellcode hinterlegten Broker hergestellt. Die Kommunikation erfolgt dabei über WebSockets[9], sodass keine zusätzliche Konfiguration im Browser erforderlich ist. Direkt zu Beginn generiert der Client außerdem eine zufällige Kennung (`clientId`), um sich gegenüber dem Broker eindeutig zu identifizieren. Da der Webclient im Gegensatz zu einem Roboter keine feste Identität benötigt und nur für die Dauer der aktuellen Browsersitzung aktiv ist, spielt es keine Rolle, ob diese ID bei einem erneuten Laden der Seite eine andere ist.

Nach erfolgreicher Verbindung abonniert der Client drei zentrale Topic-Muster:

- `+/+/register` – empfängt Registrierungsdaten aller im Netzwerk verfügbaren Roboter, einschließlich Typ, ID und unterstützter Aktionen.
- `+/+/status` – liefert Statusmeldungen (online/offline, gesperrt/frei) und aktualisiert so die Geräteliste in Echtzeit.
- `+/+/control/acknowledge` – empfängt die Bestätigung eines erfolgreichen Claim-Handshakes für die exklusive Steuerung eines Roboters.

Durch diese Abonnements ist die Benutzeroberfläche unmittelbar nach dem Aufruf bereit, verfügbare Roboter anzuzeigen, ihren aktuellen Zustand zu visualisieren und auf Interaktionen der Nutzerin oder des Nutzers zu reagieren.

Kommunikation mit dem Broker:

Alle eingehenden MQTT-Nachrichten auf den zuvor abonnierten Topics werden zentral in einem einzigen Listener verarbeitet. Hierfür wird im Webclient die Ereignisbehandlung des MQTT-Clients genutzt. Innerhalb dieser Callback-Funktion wird zunächst die empfangene Nachricht in einen String umgewandelt. Anschließend erfolgt die Auswertung anhand des Topic-Pfads:

```
client.on('message', (topic, msgBuf) => {
  const msg = msgBuf.toString();
  const parts = topic.split('/');
  const [type, id, leaf] = parts;

  if (leaf === 'register') {
    // [...]
  }
  if (leaf === 'status') {
    // [...]
  }
  if (leaf === 'control' && parts[3] === 'acknowledge') {
    // [...]
  }
});
```

- **Registrierung (+/+/**register**):** Verarbeitet die von EasyMQTT generierte Konfigurationsdatei im Json format. Da diese als *retained* auf das entsprechende Topic gepublished wurden, werden nicht nur bei Neuverbindung eines Roboters, sondern auch direkt beim Seitenaufruf alle Konfigurationen aller Roboter im Netzwerk verarbeitet. Insbesondere werden sie in einer internen Datenstruktur gespeichert.
- **Status (+/+/**status**):** Verarbeitet Statusänderungen. Im Frontend werden diese allerdings anders interpretiert als im Backend. Statt den beiden Komponenten (*online/offline*), (*locked/free*) wird nur ein einziger der folgenden Wert angezeigt: (*online/locked/connected/offline*).
- **Acknowledge (+/+/**control/acknowledge**):** Bestätigt einen zuvor gesendeten Claim-Versuch. Wird hier "true" empfangen, beginnt der Webclient mit der Steuerung des Roboters und startet die Ping-Schleife. Der genaue Ablauf dieses Handshakes ist in Abschnitt 3.2.2 beschrieben.

In jedem von diesen Fällen wird am Ende die Funktion `rebuildList()` aufgerufen, die in der Gui die Roboterliste aufbaut und updated.

Für ausgehende Nachrichten ist die Methode `sendEvent(actionTopicSuffix, action)` zuständig. Sie übernimmt die Formatierung der Payload (inklusive Controller-ID) und veröffentlicht die Nachricht auf dem entsprechenden `<Typ>/<ID>/action/<Aktionsname>`-Topic:

Dynamische Anpassung der Seite:

Die Steueroberfläche passt sich zur Laufzeit an die vom Roboter gemeldeten Aktionen an. Zentral ist dabei `configureControls(actions)`, das auf Basis der `actions`-Liste das D-Pad aktiviert bzw. deaktiviert, Hotkeys mappt und Zusatzfunktionen als Buttons unter dem D-Pad aufbaut. Zu Beginn werden Key-Maps und gedrückte Tasten zurückgesetzt:

```
function configureControls(actions = []) {  
  // 1) Key-Map & aktive Keys zurücksetzen  
  keyMap = {};  
  Object.keys(activeKeys).forEach(k => activeKeys[k] = false);
```

Das D-Pad wird über eine kompakte Konfiguration beschrieben. Jede Richtung besitzt einen CSS-Selektor, ein erwartetes Action-Suffix und die zugehörigen Tastatur-Shortcuts. Der Stop-Button nutzt, wenn vorhanden, die Aktion `dir-center`, sonst `dir-stop`:

```
// 2) D-Pad Buttons konfigurieren  
const dpadCfg = {  
  forward: { sel: '.btn-up',    act: 'dir-forward',  keys: ['ArrowUp', 'w'] },  
  back:    { sel: '.btn-down',  act: 'dir-backward', keys: ['ArrowDown', 's'] },  
  left:    { sel: '.btn-left',  act: 'dir-left',    keys: ['ArrowLeft', 'a'] },  
  right:   { sel: '.btn-right', act: 'dir-right',   keys: ['ArrowRight', 'd'] },  
  stop:    { sel: '.btn-stop',  act: actions.includes('dir-center') ?  
                                'dir-center' : 'dir-stop',      keys: [' '] }  
};
```

Pro Richtung wird geprüft, ob die Aktion in `actions` enthalten ist. Falls ja, wird der Button aktiviert, das gewünschte Topic-Suffix in `data-cmd` hinterlegt und die angegebenen Tasten im `keyMap`-Objekt registriert. Nicht unterstützte Richtungen werden visuell deaktiviert. Event-Handler werden pro Button nur einmal gebunden:

```

Object.values(dpadCfg).forEach(cfg => {
  const btn = document.querySelector(cfg.sel);
  if (!btn) return;

  if (actions.includes(cfg.act)) {
    btn.classList.remove('disabled');
    btn.dataset.cmd = cfg.act;
    cfg.keys.forEach(k => keyMap[k] = cfg.act);

    // Event-Handler nur einmal anbinden
    if (!btn.dataset.bound) {
      addBtnHandlers(btn);
      btn.dataset.bound = '1';
    }
  } else {
    btn.classList.add('disabled');
    delete btn.dataset.cmd;
  }
});

```

Zusatzfunktionen folgen dem Namensschema `extra-<taste>-<funktionsname>`. Dar-
aus werden Hotkey und Label abgeleitet, als Button gerendert und ebenfalls in `keyMap`
eingetragen:

```

extraActions.innerHTML = '';
actions.filter(a => a.startsWith('extra-')).forEach(a => {
  const parts = a.split('-');
  if (parts.length < 3) return; // Format prüfen
  const hotKey = parts[1].toLowerCase();
  const label = parts.slice(2).join('-');

  // Hotkey in die Key-Map eintragen
  keyMap[hotKey] = a;

  // Button erzeugen und anhängen
  const btn = document.createElement('button');
  btn.className = 'btn btn-secondary';
  btn.dataset.cmd = a;
  btn.textContent = label || hotKey.toUpperCase();
  btn.title = `Taste: ${hotKey.toUpperCase()}`;
  addBtnHandlers(btn);
  extraActions.appendChild(btn);
});
}

```

Damit entsteht aus der vom Broker gelieferten `actions`-Liste eine vollständig dynami-
sche Oberfläche: nicht unterstützte Richtungen sind ausgegraut, Tastatursteuerung (Pfei-
le/WASD und definierte Hotkeys) ist automatisch gemappt und zusätzliche Funktionen
stehen als Buttons unter dem D-Pad bereit. Alle Interaktionen publizieren anschließend
über die einheitliche Sende-Funktion `sendEvent(...)`.

3.4 Netzwerk und Hardware

Alle für die Fernsteuerung vorgesehenen Roboter nutzten einen Arduino MKR1000[10] mit integriertem WiFi-Modul. Ursprünglich war geplant, die Bibliothek auch für ESP-basierte Controller nutzbar zu machen. Insbesondere der preislich attraktive ESP8266[11] konnte jedoch – selbst nach einer Migration des Codes auf die ESP-Architektur – die Bibliothek nicht ausführen. Aus Zeitgründen wurde dieses Problem nicht weiter untersucht. Erste Analysen deuten jedoch darauf hin, dass eine speichereffizientere und ressourcenschonendere Implementierung notwendig wäre, um den Betrieb auf Controllern mit begrenztem RAM zu ermöglichen.

Der MQTT-Broker und der Webserver wurden während der Entwicklung auf einem Windows-PC gehostet. Grundsätzlich wäre der Einsatz eines Einplatinencomputers wie eines Raspberry Pi möglich gewesen, für die Anforderungen im Projekt erwies sich die genannte Variante jedoch als am praktikabelsten. Der Broker wurde über den folgenden Befehl gestartet:

```
mosquitto -v -c "path/to/mosquitto.conf"
```

In der frühen Entwicklungsphase wurde ein mobiler Hotspot verwendet, über den sowohl Broker als auch Website bereitgestellt wurden. Diese Lösung war zwar schnell einzurichten, bot jedoch nur eingeschränkte Routing-Funktionen. Insbesondere vergaben viele Smartphones nach einem Neustart des Netzwerks neue IP-Adressen, sodass diese im Quellcode manuell angepasst werden mussten. Im weiteren Projektverlauf wurde daher auf einen dedizierten Router umgestellt, um stabile IP-Adressen und eine zuverlässigere Netzwerkumgebung zu gewährleisten.

Die Weboberfläche wurde lokal über den Python-Befehl

```
python -m http.server 80
```

gestartet und konnte anschließend über <Host-IP>:80 aufgerufen werden. Um den Aufruf per Hostnamen `robotcontrol.local` zu ermöglichen, wurde mDNS[12] eingesetzt. Dies erfolgte über die Bonjour-Druckdienste in Kombination mit einer Anpassung des Rechnernamens.

4 Fazit

Das Projekt war in technischer wie organisatorischer Hinsicht ein voller Erfolg. Die verfolgte Architektur mit klarer Trennung von Kommunikationslogik (EasyMQTT) und Steuerlogik im Sketch sowie das generische Webinterface führten zu einer stabilen, nachvollziehbaren und gut erweiterbaren Lösung.

Die Zusammenarbeit im Team funktionierte im Verlauf des Semesters immer besser. Das zeigte sich in der abschließenden Präsentation aus, in der alle Komponenten wie geplant zusammenliefen und die Funktionsfähigkeit des Gesamtsystems unter Beweis stellten.

Das Hauptziel, Robotik möglichst niedrigschwellig zugänglich zu machen, wurde erreicht. Durch den Einsatz kostengünstiger Materialien, reproduzierbarer Fertigungsschritte und einer Konfiguration-statt-Programmierung-Philosophie ist der Einstieg deutlich vereinfacht. Das Webinterface kommt ohne Installation aus, läuft auf gängigen Endgeräten und passt sich dynamisch an die Fähigkeiten des jeweiligen Roboters an. Insgesamt senken günstiger Preis, einfaches Setup und eine klare Dokumentation die Hürden für Nachbau und Erweiterung spürbar.

Literaturverzeichnis

- [1] OASIS MQTT TC, *MQTT Version 3.1.1*, 2014. Adresse: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>
- [2] Eclipse Mosquitto Project, *mosquitto.conf — Configuration Manual*, 2025. Adresse: <https://mosquitto.org/man/mosquitto-conf-5.html>
- [3] N. O’Leary, *Arduino Client for MQTT (PubSubClient)*, 2025. Adresse: <https://pubsubclient.knolleary.net/>
- [4] Arduino, *WiFi101 Library — Documentation*, 2020. Adresse: <https://docs.arduino.cc/libraries/wifi101>
- [5] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” IETF, Techn. Ber. RFC 8259, 2017. Adresse: <https://datatracker.ietf.org/doc/html/rfc8259>
- [6] B. Blanchon, *ArduinoJson — Documentation*, 2025. Adresse: <https://arduinojson.org/>
- [7] Bootstrap Team, *Get started with Bootstrap*, 2025. Adresse: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- [8] MQTT.js Contributors, *MQTT.js — MQTT client for Node.js and the browser*, 2025. Adresse: <https://github.com/mqttjs/MQTT.js>
- [9] I. Fette und A. Melnikov, “The WebSocket Protocol,” IETF, Techn. Ber. RFC 6455, 2011. Adresse: <https://datatracker.ietf.org/doc/html/rfc6455>
- [10] Arduino, *MKR 1000 WiFi — Hardware Documentation*, 2025. Adresse: <https://docs.arduino.cc/hardware/mkr-1000-wifi>
- [11] Espressif Systems, *ESP8266EX Datasheet*, Version 6.3, 2023. Adresse: https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf
- [12] S. Cheshire und M. Krochmal, “Multicast DNS,” Internet Engineering Task Force (IETF), Techn. Ber. RFC 6762, Feb. 2013. Adresse: <https://datatracker.ietf.org/doc/html/rfc6762>